

	Nicola Beccaceci s360641
Stato	Completato
Iniziato	martedì, 8 settembre 2026, 11:01
Terminato	martedì, 8 settembre 2026, 11:50
Tempo impiegato	49 min. 12 secondi
Valutazione	5,8 su un massimo di 6,0 (96,7%)
Riepilogo del tentativo	1

Domanda 1

Completo

Punteggio
ottenuto 5,8
su 6,0

Rate limiter



Un server riceve richieste da più client in modo concorrente.

Allo scopo di garantire un uso equo delle proprie risorse, implementa un limitatore di richieste per client.

Tale limitatore garantisce che non vengano elaborate più di N richieste provenienti da un dato client in un intervallo di tempo W.

Se si cerca di sfiorare tale limite, il server postpone l'esecuzione della richiesta per un tempo sufficiente a garantire l'invariante dato.

Questo meccanismo viene implementato dalla `struct RateLimiter` che offre due soli metodi:

```
pub fn new(w: Duration, n: usize) -> Self
pub fn acquire(&self, id: ClientId)
```

Ad ogni richiesta ricevuta, il server provvede a identificare il client da cui proviene, ad invocare il metodo `acquire(...)`, che implementa la logica di limitazione della frequenza e, al suo ritorno, esegue la richiesta.

Requisiti:

- In base alle richieste precedenti provenienti dallo specifico client, il metodo `acquire(...)` può tornare subito, se la richiesta non viola la regola data, oppure fermarsi per un opportuno lasso di tempo senza consumare CPU, così da far rispettare i vincoli.
- Una singola istanza di `RateLimiter` è utilizzabile da più thread contemporaneamente. La struttura dati deve essere quindi opportunamente protetta.
- L'eventuale attesa di una richiesta proveniente da certo client non deve bloccare le richieste provenienti da altri client.
- Se da uno stesso client arrivano più richieste mentre la finestra temporale è già piena, occorre fare attenzione, quando essa si svuota, a sbloccare solo tante richieste quante è ampia l'apertura che si è creata.

Consegna:

Si progetti la struttura dati `RateLimiter` e si implementino i due metodi dati.

Se il codice consegnato non può essere compilato, non verrà valutato.

*La risposta a questa domanda è stata inserita tramite un'istanza Crownlabs ed è visibile sul Portale della Didattica sulla sezione **Consegna Elaborati** di questo insegnamento.*

Commento:

Passano 12 test su 12, tuttavia, allochi inutilmente una condvar per ciascun clientId, quando per attendere il trascorrere del tempo puoi semplicemente invocare `std::thread::sleep(duration)`. Inserire il Mutex all'interno di un Arc è altrettanto inutile, in quanto non hai un possesso da condividere con altri.

Una possibile versione più economica del tuo codice è:

```
pub mod rate_limiter {

    use std::{collections::HashMap, os::unix::raw::time_t, sync::{Arc, Condvar, Mutex}, time::{Duration, Instant}};

    #[derive(Debug, Clone, Copy, Eq, PartialEq, Hash)]
    pub struct ClientId(usize);

    impl ClientId {
        pub fn new(id: usize) -> Self {
            ClientId(id)
        }
    }

    pub struct RateLimiterState {
        // uso una Hashmap in modo tale da ottenere le richieste passate con complessità (quasi) costante O(1)
        clients: HashMap<ClientId, Vec<Instant>>
    }

    impl RateLimiterState {
        pub fn new () -> Self {
            Self {
                clients: HashMap::new()
            }
        }
    }

    pub struct RateLimiter {
        inner: Mutex<RateLimiterState>, // (mutex, cvar_threads)
        duration: Duration,
        requests_threshold: usize
    }

    impl RateLimiter {
        pub fn new(d: Duration, n: usize) -> Self {
            Self {
                inner: Mutex::new(RateLimiterState::new()),
                duration: d,
                requests_threshold: n
            }
        }
    }
}
```

```

pub fn acquire(&self, id: ClientId) {
    let mutex = &self.inner;
    loop {
        let mut guard = mutex.lock().unwrap();

        if let Some(requests) = guard.clients.get_mut(&id) { // sono già presenti richieste passate
            // aggiornare il vettore delle richieste attive filtrando le richieste non più attive
            // ovvero le richieste che non appartengono all'intervallo [Instant::now(), Instant::now() -
self.duration]

            let lower_bound = Instant::now() - self.duration;
            let active_requests:Vec<Instant> = requests.iter().filter_map(|&instant_request| {
                if instant_request > lower_bound {
                    Some(instant_request)
                }
                else {
                    None
                }
            }).collect();
            *requests = active_requests;

            // se il vettore delle richieste attive contiene almeno N elementi, attendo che si svuoti (eseguo
un'altra iterazione)
            if requests.len() >= self.requests_threshold {
                let time_first_request = *requests.first().unwrap();

                // tempo da aspettare = tempo ultima richiesta + duration - Instant::now()
                let time_to_wait = (time_first_request +
self.duration).saturating_duration_since(Instant::now());
                drop(guard);
                std::thread::sleep(time_to_wait);
            }
            else { // altrimenti inserisco la richiesta
                requests.push(Instant::now());
                return;
            }
        }
        else { // prima richiesta
            guard.clients.insert(id, vec![Instant::now()]);
            return;
        }
    }
}

```